

Approach

The answer is **always computed by a deterministic symbolic solver**, never by a language model. The LLM (**Qwen2.5-3B-Instruct**, Qwen License, $\leq 8B$, self-hosted via Ollama, OpenAI-compatible /v1/*) does three utility roles only: (1) extract physical quantities from prose when regex misses, (2) translate a natural-language claim into FOL for Z3, (3) phrase the explanation. Every numeric result comes from SymPy/pint; every logical verdict from forward-chaining or a Z3 proof. The explanation is generated from the solver’s own trace, so it cannot diverge from the answer – directly serving P2 (explanation) and P3 (reasoning depth).

Pipeline

- **Router**: presence of **premises-NL** -> logic, else physics.
- **Physics**: cleaner -> topic classifier (ID-prefix + keyword + field-vs-force routing guard – symbol fallback) -> regex extractor (LLM fallback with $N=3$ self-consistency vote + TF-IDF few-shot RAG over solved train rows) -> pint unit normalisation -> **30-formula YAML registry** (capacitor energy/Q/U, Coulomb single/midpoint/perp-bisector/equilateral, parallel-plate air & dielectric, electric field from force / point-charge / 2-charge midpoint, magnetic flux/solenoid/wire, RLC impedance/resonance/factor/power, power factor) -> SymPy compute -> sanity verifier -> trace explanation.
- **Logic**: TF-IDF premise selector ($R@5 = 81.3\%$ vs the dataset’s gold **idx** field) -> rule parser -> forward chainer -> **Z3** entailment (multi-variable universals, comparison literals) with a NL->FOL parser/vocabulary rejection-and-retry loop (Slide-27 endorsed) -> per-option entailment for MC with **Unknown** abstention.
- **Self-correction** ON: solver re-checks the LLM draft, feedback is *solver-grounded* (never invents numbers/premises), ≤ 2 rounds. Output schema {**answer**, **explanation**, **cot**, **premises**, **fol**, **confidence**}; malformed JSON impossible (Pydantic); unreachable LLM degrades to the deterministic answer (never a 500).

Training (ablation, not deployed)

QLoRA-SFT on Qwen2.5-7B (unsloth/qwen2.5-7b-instruct, rank 16 alpha 32, 2 epochs on free Colab T4, 1,765 train pairs from official release 2026-05-09). Measured on a **163-row SFT-unseen** holdout of the 2026-05-15 release (built to fix a 99% leakage in our prior holdout – `scripts/build_sft_unseen_holdout.py`): single SFT-7B = 24.5% (-6.6pp LD regression, 5x latency); hybrid LD->3B + rest->SFT-7B = 30.1% confirmed. **Not deployed**: Q3 “one model loaded at any moment” compliance + vLLM-LoRA stack is 1-2 days of integration risk that doesn’t justify the marginal lift over deterministic 27.6%.

Data & compliance

Only the official EXACT2026 release **2026-05-15** is used. No external data, no synthetic from closed-source models, no crawled data, no GPT/Claude/Gemini in the pipeline. See `docs/data_disclosure.pdf`. 253 automated tests; ruff + mypy CI; 26 ADRs documenting every architectural decision and measurement.

Internal results (163-row SFT-unseen clean holdout, frozen unit-/round-aware scorer)

Variant	Physics Full	Logic correct
Rule-only (no LLM, Day-1)	0.8%	35.9%
Honest baseline on official data (Day-21)	22.1%	22.2%
+ Day-23 F1+F2: 6 audit-driven formulas + field-vs-force routing guard	27.6%	–
+ F3 TF-IDF premise selector / E5 NL->FOL rejection loop	–	23.5%
+ Day-23 hybrid (LD->3B, rest->SFT-7B)	30.1% (ablation)	–

Physics correctness uses strict exact value-and-unit match (pint dimension + 1% rel tolerance OR round-aware match at the gold’s stated precision). The deterministic solver is exact where extraction succeeds; the residual is LLM extraction recall on hard multi-step problems. Logic is translator-bound on the 3B; the symbolic Z3 core is sound.

Deployment

`docker compose -f docker/docker-compose.yml up -d ->` GPU Ollama sidecar (qwen2.5:3b pinned resident, `OLLAMA_KEEP_ALIVE=-1`) + CPU FastAPI solver; always-on with `/healthz`, cron pre-warm (`scripts/prewarm_cron.sh`). Reproduce: `uv run pytest -q; uv run python scripts/run_eval.py --task {physics,logic} --with-llm --split data/official_v20260515/eval_split/<task>_eval.jsonl`.